Generative Modelling Case Study Using Generative Adversarial Networks

Name: Revanth Kumar Sagala

Student ID: 23076874

GitHub Repository

1. Introduction

The focus of this report is on generative modelling which, in unsupervised machine learning, describes the process of learning the data's probability distribution enabling the creation of artificial data similar to the real data sample used to create the model, or a related variable. Within generative modeling, there is data generation and density estimation which relates to determining the probability density function, and in latent space models and transformation from latent space to complex data representations are central themes in order for them to work as an image-generating agent that understands underlying structures within the image. Important areas of work include the development of generative network architectures such as the generative adversarial network (GAN), autoregressive models, and autoencoders. All have varied application to many tasks, such as simulation, photo generation and text creation. We evaluate GAN's on several tasks and measure the resulting models with both visual assessment and statistical metrics.

2. Aim

The objective of this case study is to research the effectiveness of Generative Adversarial Networks on varying data domains. This report consists of four tasks; learning simple artificial data (reproduce the 2D sin wave), learning complicated artificial medical data (BloodMNIST data), learning artificial traffic security data and learning artificial hand drawn data (Quickdraw pizza data). In order to assess the effectiveness of our model, both qualitative visual assessments and quantitative measurements (learning loss, PCA visualization, FID measures) are used.

3. Methodology

Generative Adversarial Networks (GANs) are deep learning models designed to generate new data samples based on the distribution of existing data. GANs are comprised of two neural networks simultaneously trained: a generator and a discriminator. A generator maps a latent noise vector into synthetic samples., the discriminator evaluates the samples to identify whether they are real samples (from the training data) or are fake (samples from generator). The two networks compete in an adversarial process, where the generator aims to fool the discriminator and the discriminator aims to correctly distinguish real from generated data. These models continue with that kind of competition that a discriminator enhances its capability to learn the differentiation, and a generator produces better fake samples. Through, this competition the model helps to extract out the intrinsic properties of the data and creates much better models as far as generator is concerned; such that after training, it would be difficult to recognize their output from the actual training sample. The training of GANs could prove very hard, and there can be many related problems. Some examples include sensitivity to hyper parameters,

convergence issue, instability, and the problem of “mode collapse”. To tackle the problems, we use techniques like normalization within the training batch (batch normalization), carefully selected values of learning rate, modifications of the model architecture, a balanced development, especially balancing training for the generator and discriminator. We should then check how well trained is the model; we can check qualitatively for whether samples are truly indistinguishable from the original one and can use standard evaluation metrics for how well the two sets are matched distributionally.

4. Part 1: GAN on Synthetic 2D Data

We initially applied Generative Adversarial Networks on a synthetic two-dimensional dataset to examine their behaviour under a simplified training condition. This allowed comparison between synthetic and real data, along with observations about adversarial training, instability and how the models mode behaviour evolved during the process.

4.1 Sine-wave GAN Reproduction

In the first experiment we reproduced the tutorial GAN using a sine-wave dataset, where samples followed a nonlinear sinusoidal pattern. A fully connected generator and discriminator were trained on the dataset. After training, the generated samples resembled the general shape of the sine curve, showing that the GAN was able to learn a continuous nonlinear distribution from noise inputs.

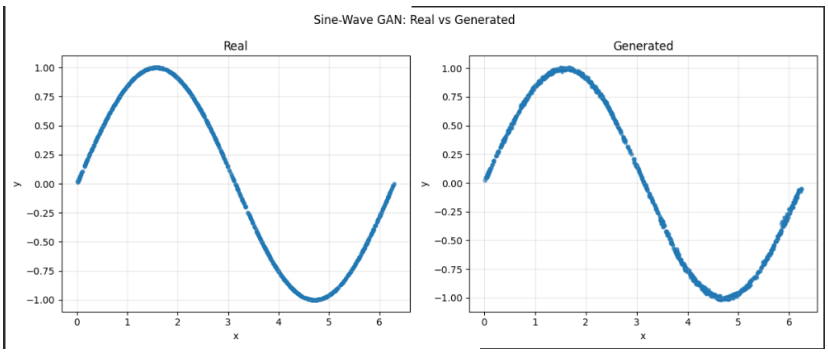

Fig 4.1. Real vs Generated Sine-wave Scatter Plot

4.2 Custom 2D Distribution: Mixture of Gaussians

For the second experiment we used a mixture of Gaussian clusters as a more difficult multi-modal dataset. Instead of just modelling one continuous data mode like we were doing in the sine-wave case, the generator had to model multiple independent data modes at the same time in this case. This meant it was significantly more complex to force the generator to produce samples near the modes while being unaware of where the un-generated samples are within those clusters and while accurately modelling both the shape of those clusters and their centre points simultaneously. From our result we can see the GAN had managed to learn the data well and produced samples near the cluster locations, but that there were some overlaps between neighbouring clusters and uneven density in samples. The conclusion is that GANs can capture multiple mode data sources even though this task is considerably more difficult than modelling a single continuous mode data source.

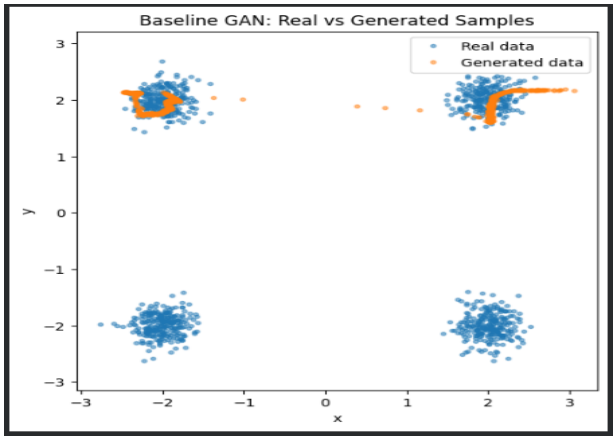

Fig.4.2. Real vs Generated Gaussian Mixture Scatter Plot (baseline GAN)

4.3 Architecture Modification and Comparison

We altered the original GAN framework by deepening its layers, including several batch normalization layers to enhance its performance. As a result, the new model produced more clearly separated clusters and generated more consistent results over epochs when compared to baseline GAN. This experiment validates that parameters like the depth of the GAN model's layers and its training regime dramatically affect its outcomes.

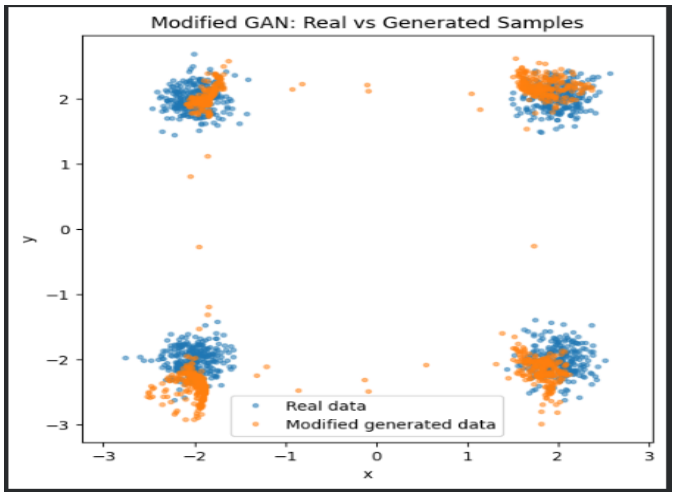

Fig. 4.3. Real vs Generated Gaussian Mixture Scatted plot (modified GAN)

5. Part 2.1: Blood Cell Microscope Imaging with BloodMNIST

The primary purpose of this task was to explore whether a Generative Adversarial Network (GAN) could create realistic synthesized medical images. We utilized the BloodMNIST dataset which comprises labelled microscope images of a range of blood cells. For this task, we employed a Deep Convolutional Generative Adversarial Network (DCGAN) to train on this dataset. We chose the DCGAN architecture as the convolution layers were well-suited to learning spatial features like colour patterns, textures, and cellular structures found in microscope images. The trained model successfully generated new blood cell images possessing several significant traits of the training dataset.

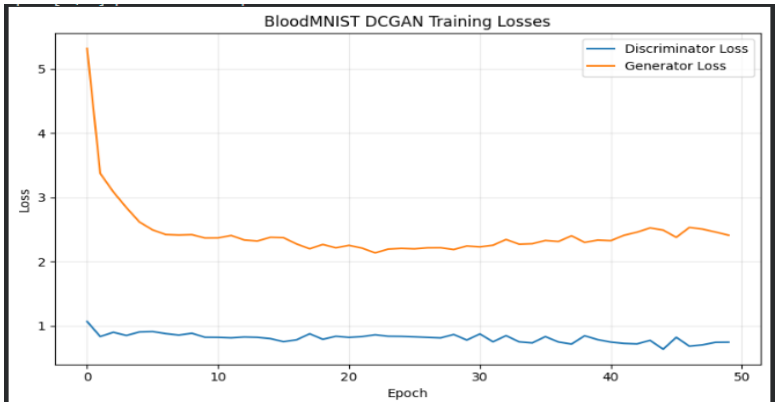

Fig.5.1. Training Losses

The generator after training successfully generated images that captured a number of attributes from the real training dataset. As shown in Figure 5.2, we can see that the produced samples from the generator have learned to reproduce similar circular cells with circular boundaries, nucleus region of purple colour and blood smear background in pale pink colour. This indicates that the model successfully learned meaningful visual features rather than producing random noise.

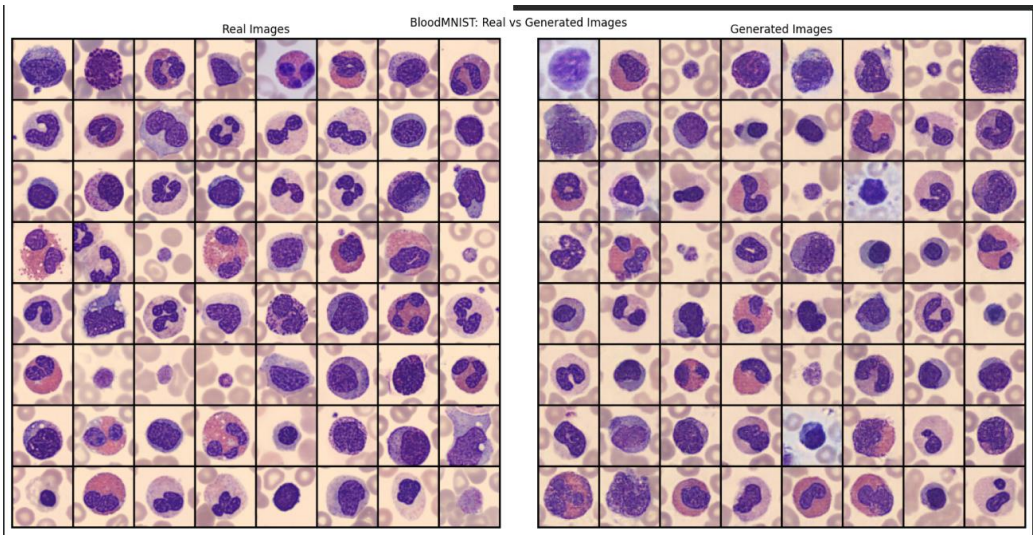

Fig.5.2. BloodMNIST: Real vs Generated Images

A quantitative assessment based on Fréchet Inception Distance generated a result of 62.42. Based on this score, there can be seen to be a moderate resemblance between the real and generated image set. However, many samples generated were convincing, but some samples were somewhat blurry, and some cell shapes were repeated for a number of times. This implies that the GAN had converged but was able to provide greater variety and better images with longer training cycles and or by utilizing more complex networks, or other types of GAN.

Part 2.1: Extension Task:

For the extension task, a conditional GAN (cGAN) model was applied to the BloodMNIST dataset for generating blood cell images per class. Instead of basic GAN, cGAN receives the class labels during training and enables the generator to generate

images for specified blood cell class. One sample per class was visualized in Figure 5.3. We could observe that by conditional information, it became easier to control the generation, making it produce specific images with requested blood cell types rather than purely randomly. Some classes appeared more realistic than others, indicating that class complexity and data variation still influenced generation quality.

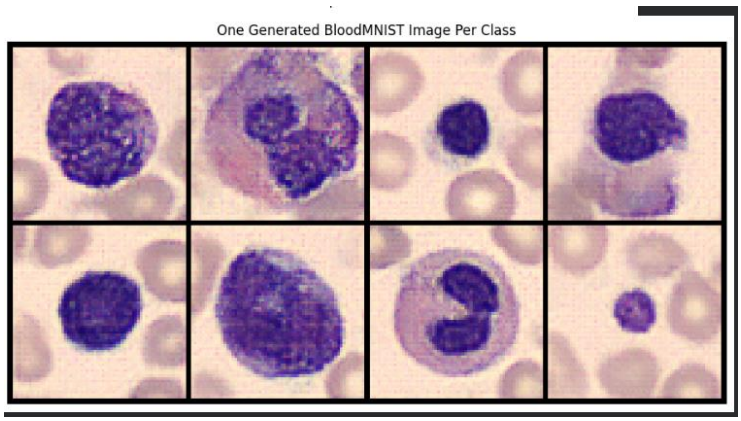

Fig.5.3. One synthetic BloodMNIST image generated for each class using the cGAN.

6. Part 2.2: Cybersecurity Traffic Generation using CICIDS2017

This task examined the capability of GAN to simulate the records of cybersecurity traffic utilizing the information of CICIDS2017 dataset. For this project, we have decided on Wednesday containing benign traffic and selected denial-of-service attack classes. As a result of the information isn't an image-based information however a totally organized numeric information, a totally linked GAN has been deployed quite than a convolutional GAN.

As an initial step before training, we pre-processed information to remove non-numeric or invalid values, filtered the classes, and scaled features. After training , we generated synthetic feature vectors and compared them with the real dataset information by using Principal Component Analysis (PCA) and visualizing their information (seen in Fig 6.2).

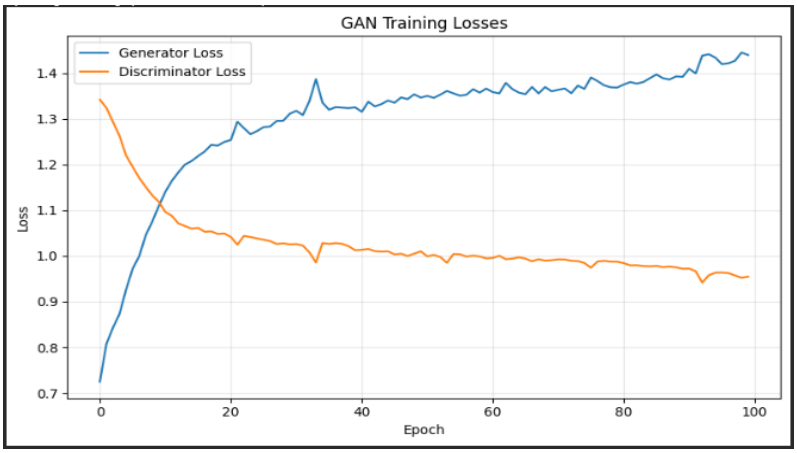

Fig. 6.1. GAN training losses

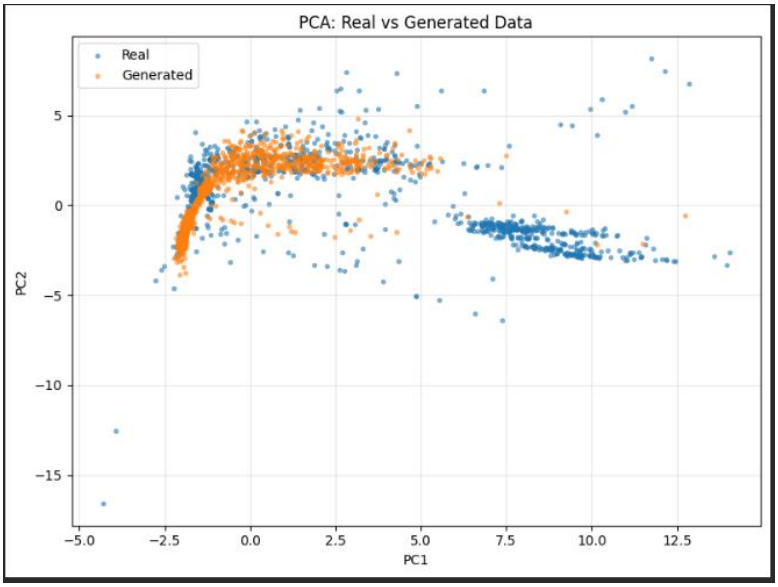

Fig.6.2. PCA comparison of real CICIDS 2017 vs Generated Data.

The PCA projection plot shows the generated samples were able to capture the high-level shape of the real data distribution, and there was a degree of overlap between real and synthesized data points, which imply that the GAN can learn relevant relationships among the network traffic features. While a separation between the distributions still exits, it's a reasonable finding, as learning complicated relationships in tabular cybersecurity data is usually much more difficult than learning images. Some possible future improvements would be extended training, feature engineering or the use of a GAN model suited for tabular data.

7. Part 2.3: QuickDraw Pizza Sketch Generation

This task employed the QuickDraw pizza subset for training a DCGAN to generate hand-drawn pizza sketches. Compared with the BloodMNIST task, the input here contains sparse line-based drawing data with large variability in style, rotation, and shape, so a convolutional GAN was employed to learn the spatial structure of sketches.

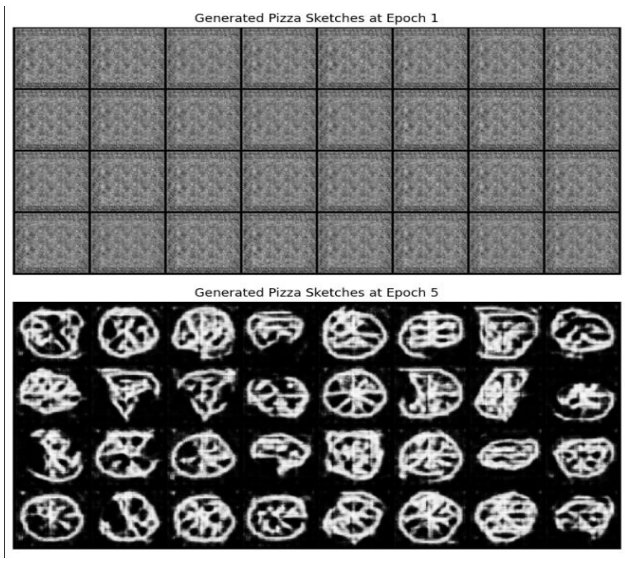

Fig.7.1: Generated pizza sketches at epoch 1 and at epoch 5

Figure 7.1, 7.2 represent training evolution of our model. In early epochs, generator generated somewhat blurry results that are lacking structure. Afterwards the circle like pizza base, edges of slices, patterns of pizza toppings slowly came in notice. By end of training, some generated samples are strongly recognizable as sketches of pizza as they captured some of crucial structural property of original data set.

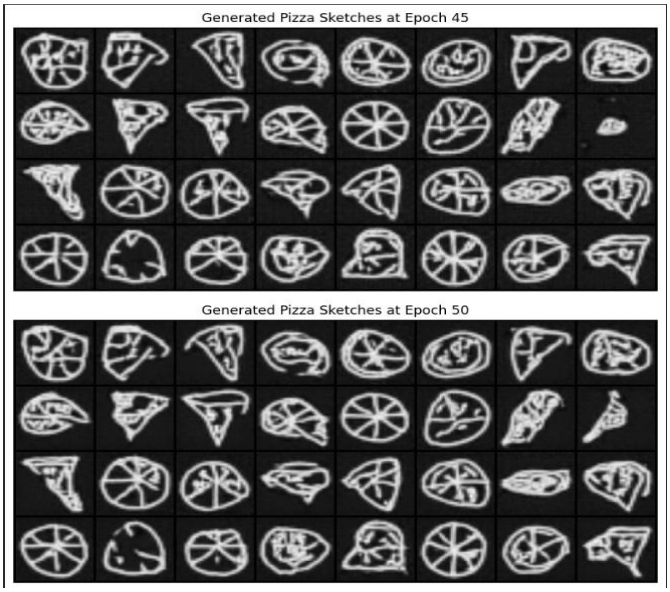

Fig.7.2: Generated pizza sketches as epoch 45 and 50

During fine-tuning, the quantitative FID (Fréchet Inception Distance) score improved significantly from approximately 111 to 65.33 after training for a higher number of epochs, indicating increased realism and better distribution matching between real and generated images. However, some generated outputs remained repetitive and certain geometric structures were still imperfect. These limitations could be further improved through longer training, larger model capacity, stronger regularisation, or more advanced GAN architectures designed for stable image generation.

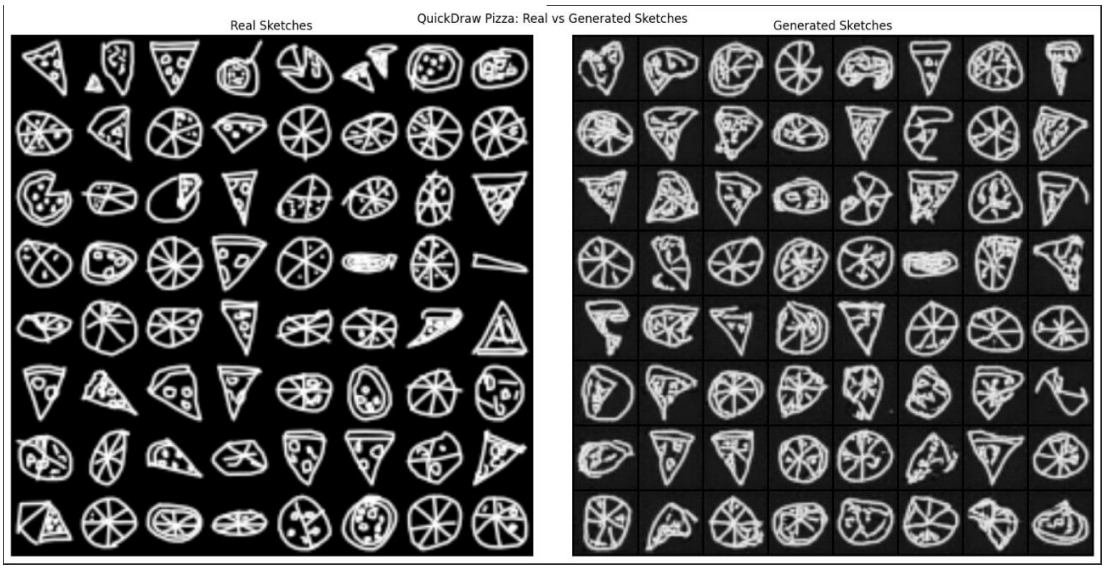

Fig.7.3: Final comparison of Real vs Generated sketches at end of 50 epochs

8. Conclusion and Future Improvements

In summary, this work demonstrated the effective application of Generative Adversarial Networks on different data and across different tasks in practice: synthesizing 2D data, generating synthetic images in medical imaging and cybersecurity, and creating images based on hand-drawn sketches.

The experiments conducted proved that GANs are able to learn complex as well as simple data distributions and to create realistic outputs when sufficiently trained. For synthetic data, clear insights into adversary training dynamics are provided and architecture alterations have been presented which could aid with quicker convergence and higher output quality.

For real datasets, GANs were found to learn complex feature patterns: BloodMNIST produced realistic microscope-style blood cell images with meaningful cellular structure, CICIDS2017 data set was modelled faithfully to learn complex patterns in network traffic features and the resulting pizza sketch images were recognizable enough for QuickDraw dataset. These experiments also highlight GAN's versatility to different types of data. There were still, however, multiple limitations observed such as training instability, occurrence of mode collapses, repetitive samples and blurriness.

The quality and performance depended strongly on architecture and hyperparameter selection and training epoch number. The techniques which could be used to mitigate these shortcomings in the future included training the Wasserstein GAN, using a more sophisticated GAN architecture like StyleGAN, increasing training budget and using stronger evaluation metrics. The conditional generation can also be further utilized in order to strengthen controllability of generation for specific classes.

Overall, the study showed that GANs remain a powerful generative modelling framework, while also requiring careful tuning and critical evaluation to achieve reliable results.

9. References

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) *Generative Adversarial Nets*. Advances in Neural Information Processing Systems, 27,. Link

2. Radford, A., Metz, L. and Chintala, S. (2016) *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. Link

3. Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B. and Bharath, A.A. (2018) *Generative Adversarial Networks: An Overview*. IEEE Signal Processing Magazine, 35(1), Link

4. Bond-Taylor, S., Leach, A., Long, Y. and Willcocks, C.G. (2021) *Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models*. Link

5. Xu, L., Skoularidou, M., Cuesta-Infante, A. and Veeramachaneni, K. (2019) *Modeling Tabular Data using Conditional GAN*. Advances in Neural Information Processing Systems, 32. Link

6. Karras, T., Laine, S. and Aila, T. (2019) *A Style-Based Generator Architecture for Generative Adversarial Networks*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Link

7. Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H., Ni, B. and Gao, Y. (2023) *MedMNIST v2: A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification*. Scientific Data, 10, 41. Available at: https://www.nature.com/articles/s41597-022-01721-8

8. MedMNIST (2024) *BloodMNIST Dataset* (dataset). Link

9. Isola, P., Zhu, J.-Y., Zhou, T. and Efros, A.A. (2017) *Image-to-Image Translation with Conditional Adversarial Networks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Link

10. Canadian Institute for Cybersecurity (2017) *CICIDS2017 Dataset* (dataset). Kaggle Link, Website Link

11. Google Creative Lab (2018) *Quick, Draw! Dataset* (dataset). Website Link, GitHub link

12. Mirza, M. and Osindero, S. (2014) *Conditional Generative Adversarial Nets*. arXiv preprint arXiv:1411.1784. Link

13. PyTorch (2024) *PyTorch Documentation*. Link